
L02 – Awk, Cut, Paste, and Join

0. Some useful unix-y information.

Most unix shells remember your history. So, if you type:

```
>> history
```

you see a list of the commands you've typed. Often (especially in this exercise) you may want to type a command that is similar to one you've already typed. In this case, there are ways to get back to the command you typed previously to edit it. The easiest way is through the up arrow. Just hit this and it will give you your previous commands.

If you want to rerun a previous command, say `ls`, you can type

```
!1
```

which will execute the last command that has "l" in it.

Another way is to type `history`, and note that there's a number next to each command you've typed. Then you can type

```
>> ![number]
```

to reexecute this command. Also

```
>> !!
```

executes the last command you typed.

1. Awk

In the geosciences, we often have text-based tables of information we need to process. Like many things in unix, there are multiple ways to approach extracting data from tables, and one widely used way is `awk`.

Awk is a programming language. However, in the geosciences it is typically used on the command line to process text-based data. The name awk, comes from its authors names: Alfred **A**ho, Peter **W**einberger, and Brian **K**ernighan.

This lecture is aimed at giving you a basic working knowledge of awk. This document should just be viewed as an awk primer, for more info on all the things you can do with awk there are a ton of amazing resources available on the web. To get started let's create a simple example file to play around with.

Let's say we want to know all of the earthquakes that have occurred in Colorado since the year 2000. To keep the file size small, let's only look for earthquakes with magnitude ≥ 3.5 .

To do this, we need to get data from the IRIS Data Management Center (DMC): <http://www.iris.edu/dms/nodes/dmc/>. There are a number of ways to do this via web forms that one can fill out, or perl functions, or find_events in SOD, etc. (<http://www.iris.edu/dms/nodes/dmc/data/#access>). We'll use a way that creates a URL (the web address you can type in a browser window) that queries the IRIS computers to get the data. Here's a webpage explaining some details about the format: <http://service.iris.edu/fdsnws/event/1/>. Normally, one might want to type the request into a browser, and have the results displayed there. But let's say we want to do this from the command line, and skip the whole browser thing. We can do this through the **wget** command, which is a command line unix program that can retrieve data from web pages.

we can type

```
wget -qO - "URL"
```

To get the data we need, we type the command:

```
wget -qO - "http://service.iris.edu/fdsnws/event/1/query?starttime=2000-01-01&format=text&minmag=4.0&minlatitude=37&maxlatitude=41&minlongitude=-109&maxlongitude=-102" > colo_eq.txt
```

-q says use **wget** in quiet mode

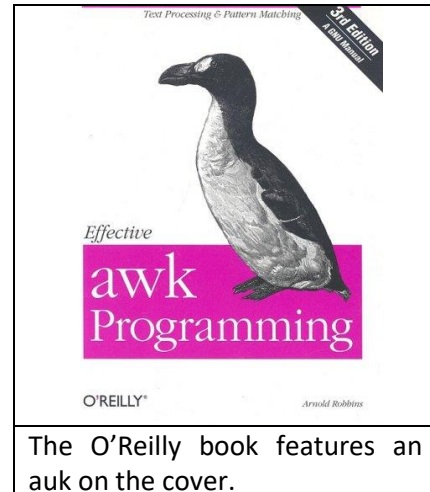
-O - says output to the terminal (standard out).

> says take the output and put it into a new file called colo_eq.txt.

(if this command doesn't work for you, it could be because you cut and pasted it, and the quote character " may need to be specifically typed in the command line).

Type this to look at the resulting file

```
>> cat colo_eq.txt
```



or this

```
>> tail -2 colo_eq.txt
```

or this

```
>> head -2 colo_eq.txt
```

or this (hit "q" to stop this):

```
>> less colo_eq.txt
```

In awk we refer to each line in a file as a record, and each column as a field. In the above example, the first record is a bunch of headers, and then second line is some actual data for a magnitude 3.9 earthquake. Often fields are separated by spaces or commas, but in this case they are separated by this character: "|". We can tell awk about the | separator by invoking awk with the field separator flag, eg.

```
>> awk -F "|" "
```

Awk works by scanning through each line of text (or record) in the file and carrying out any instructions you tell it on that line. On the command line, instructions are enclosed in single quotes and take the format

```
>> awk 'pattern { action }' file
```

We can make the awk command more than one line long too:

```
>> awk '/ search pattern1/ {Actions}
      / search pattern2/ {Actions}' file
```

The pattern part of the awk command line often is used to tell awk to search for a particular pattern before it commits to action. The action is often

In awk we access fields using syntax like: **\$1** or **\$2**. **\$1** indicates that you are referring to the first field or first column. **\$0** means print all fields.

Example 1 - Printing fields:

What is the output for the following examples?

```
>> awk -F "|" '{print $2}' colo_eq.txt
>> awk -F "|" '{print $3, $4}' colo_eq.txt
>> awk -F "|" '{print "lat="$3,"lon="$4}' colo_eq.txt
>> awk -F "|" '{print "lat="$3,"lon="$4,"magnitude="$11}'
colo_eq.txt
```

We can also do some simple arithmetic with awk.

Example 2 – Simple arithmetic on fields

```
>> awk -F "|" '{print ($3+$4)}' colo_eq.txt
>> awk -F "|" '{print int($3+$4)}' colo_eq.txt
```

The last example shows that in addition to the simple arithmetic commands, **awk** also has some useful numeric functions, such as **sin**, **cos**, **sqrt**, etc. To see the full list check out the **awk** man page.

A real useful ability is to be able to search within the files. First, let's introduce some of the variables that are built into **awk**:

awk Variable name	What it stands for
FILENAME	Name of current input file
RS	Input record separator (Default is new line)
OFS	Output field separator string (Blank is default)
ORS	Output record separator string (Default is new line)
NF	Number of fields in input record
NR	Number of input record
OFMT	Output format of number
FS	Field separator character (Blank & tab is default)

These may not all make sense right now, but we'll come back to some of them later. As an example, you may have noted that the first line we are printing has the column (field) headers. This is annoying, so let's get rid of it.

Example 3 – Simple sorting routines

Try these examples on for size:

```
>> awk -F "|" 'NR > 1 {print
"lat="$3,"lon="$4,"magnitude="$11}' colo_eq.txt
```

What if we want to only look at earthquakes with magnitudes > 5?

```
>> awk -F "|" ' $11>5 {print "lat="$3,"lon="$4,"magnitude="$11}'
colo_eq.txt
```

What if we want to look only at earthquakes with locations from the National Earthquake Information Center (NEIC)? Easy, we do a string match:

```
>> awk -F "|" ' $6 ~ /NEIC/ {print
"lat="$3,"lon="$4,"magnitude="$11}' colo_eq.txt
```

What if we don't like the NEIC, and want everything but what they do. The "!" means "not".

```
>> awk -F "|" ' $6 !~ /NEIC/ {print
```

```
"lat="$3,"lon="$4,"magnitude="$11}' colo_eq.txt
```

The comparison operators that awk allows are:

<	Less than.
< =	Less than or equal.
= =	Equal.
!=	Not equal.
> =	Greater than or equal.
>	Greater than.
~	Contains (for strings)
!~	Does not contain (strings)

To make things even more interesting we can add some logic to our conditionals! In the following examples **&&** is the **AND** operator and **||** is the **OR** operator.

Example 4 – sorting with logic

```
>> awk -F "|" ' $6 ~/NEIC/ && $11>5 {print
"lat="$3,"lon="$4,"magnitude="$11}' colo_eq.txt

>> awk -F "|" ' $3 > 40 || $11>5 {print
"lat="$3,"lon="$4,"magnitude="$11}' colo_eq.txt
```

You can also specify that awk does something either before starting to scan through the file (**BEGIN**) or after awk has finished scanning through the file (**END**).

Example 5 – BEGIN and END

```
>> awk -F "|" \ END {print NR} \ colo_eq.txt
>> awk -F "|" \ BEGIN {print NF} \ colo_eq.txt
awk -F "|" 'BEGIN {print "Example #5"} NR>1 {print $3,$4}'
colo_eq.txt
```

You can also set variables in awk and do operations with them. Occasionally it comes in handy.

Example 6 – awk variables

Here's a quick example that sets a variable `x = 1` at the beginning and increments the variable by one at each record, printing the variable out as a new field for each record.

```
awk -F "|" ' BEGIN {x=1} NR>1 {print "Earthquake_"x++,
$3,$4,$5,$11}' colo_eq.txt
```

The following table might help to make the above example a little more transparent.

Assignment operator	Use for	Example	Equivalent to
<code>+=</code>	Assign the result of addition	<code>a += 10</code> <code>d += c</code>	<code>a = a + 10</code> <code>a = a + c</code>
<code>-=</code>	Assign the result of subtraction	<code>a -= 10</code> <code>d -= c</code>	<code>a = a - 10</code> <code>a = a - c</code>
<code>*=</code>	Assign the result of multiplication	<code>a *= 10</code> <code>d *= c</code>	<code>a = a * 10</code> <code>a = a * c</code>
<code>%=</code>	Assign the result of modulo	<code>a %= 10</code> <code>d %= c</code>	<code>a = a % 10</code> <code>a = a % c</code>

In example #3, we showed an example of using `awk` with a conditional.

```
>> awk -F "|" ' NR > 1 {print "lat="$3,"lon="$4,"magnitude="$11}'
colo_eq.txt
```

Essentially, this example states:

If the record number is greater than 1 then print out the fields 3, 4, and 11. `Awk` also supports a syntax with `if` statements. E.g.,

this

```
>> awk -F "|" ' NR > 1 {print "lat="$3,"lon="$4,"magnitude="$11}'
colo_eq.txt
```

is equivalent to this

```
>> awk -F "|" ' {if (NR>1) print
"lat="$3,"lon="$4,"magnitude="$11}' colo_eq.txt
```

Note that this time the `NR>1` is inside the curly brackets. It is sometimes very useful to also have an `else` or `else if` statement to play around with. The next couple of examples show how to do this.

Example 7 – Control structures

```
awk -F "|" 'NR>1 {if ($11>5) print "big earthquake"; else if
($11<4.5) print "puny earthquake"}}' colo_eq.txt
```

```
awk -F "|" 'NR>1 {if ($11>5) print "big earthquake"; else if
($11<4.5) print "puny earthquake"; else print "just right
earthquake"}}' colo_eq.txt
```

Using the command **printf** it is possible to format the output from **awk**. **Printf** is essentially the same as that in C. You define the width of the column, whether to left or right justify and the type of information that will be outputted—such as a string, floating point, or decimal number.

Example 8 – Formatted Output

```
awk -F "|" 'NR > 1 {print $1,$2,$3,$4,$5}' colo_eq.txt

awk -F "|" 'NR > 1 {printf ("%10s %10s %-7.4f %-7.4f \n",
$1,$2,$3,$4,$5)}' colo_eq.txt
```

What if we want to format something coming from another command on the fly, rather than making a table? To do this, we can use something really cool in unix, the pipe “|”. This says, take the output from the program on the left of the pipe, and send it to the program on the right. When we write shell scripts, we’ll find this leads to a lot of possibilities.

For instance, we can take the output of the **wget** command and pipe it to **awk**:

```
>> wget -qO -
"http://service.iris.edu/fdsnws/event/1/query?starttime=2000-01-
01&format=text&minmag=4.0&minlatitude=37&maxlatitude=41&minlongit
ude=-109&maxlongitude=-102" | awk -F "|" 'NR>1 {print
$3,$4,$5,$11}'
```

Now, say we want to take the output of this and put in it a file. Then we add a redirect “>”.

```
>> wget -qO -
"http://service.iris.edu/fdsnws/event/1/query?starttime=2000-01-
01&format=text&minmag=4.0&minlatitude=37&maxlatitude=41&minlongit
ude=-109&maxlongitude=-102" | awk -F "|" 'NR>1 {print
$3,$4,$5,$11}' > earthquakes.txt
```

Or, we may want to append this output to an existing file. Then use a “>>”.

```
>> wget -qO -
"http://service.iris.edu/fdsnws/event/1/query?starttime=2000-01-
01&format=text&minmag=4.0&minlatitude=37&maxlatitude=41&minlongit
ude=-109&maxlongitude=-102" | awk -F "|" 'NR>1 {print
$3,$4,$5,$11}' >> earthquakes.txt
```

This will make a new file if earthquake.txt doesn’t exist, and append the output of the command to the bottom of the file if earthquake.txt exists.

Let’s sort these by magnitude before we put this in earthquakes.txt. To do this we pipe the output of **awk** to the **sort** command. Note that **awk** is only outputting four fields: latitude, longitude, depth, and magnitude. To sort on the 4th field, we’ll type **sort -k4**. This will put the

largest magnitude earthquake at the bottom of the file, but let's do a reverse sort that puts it in the top using **sort -k4 -r**:

```
>> wget -qO -
"http://service.iris.edu/fdsnws/event/1/query?starttime=2000-01-
01&format=text&minmag=4.0&minlatitude=37&maxlatitude=41&minlongit
ude=-109&maxlongitude=-102" | awk -F "|" 'NR>1 {print
$3,$4,$5,$11}' | sort -k4 -r > earthquakes.txt
```

For this example, we'll write over any older earthquakes.txt file, using the redirect command (>).

2. Cut, Paste, and Join

This section describes three utilities that are often used in conjunction with awk for quickly manipulating fields in files.

Paste

Sometimes you may want to extract columns of information from different files and combine them into one file. **Paste** is the perfect utility for this.

Consider the two files:

a.txt	b.txt
a1	b1
a2	b2
a3	b3
a4	b4
a5	b5

We can combine them as follows:

```
>> paste a.txt b.txt > c.txt
```

Join

If two separate files share a common field they can be combined with join. Consider two files:

A.txt		B.txt	
Vs	7.2	Vs	6.3
Vp	11.3	Vp	12.4
Rho	6.6	Rho	5.9

Now try:


```
>> join A.txt B.txt > C.txt
```

Cut

Cut is incredibly useful for chopping up files into fields. Use the `-d` flag to specify a new delimiter, and the `-f` flag to state which fields to print out.

Consider a file as follows (`A.txt`) that uses underscores to separate fields:

Vs_7.2
Vp_11.3
Rho_6.6

One could just extract the numeric values by:

```
>> cut -d_ -f2 A.txt
```

Another place I find cut useful for is in extracting information out of file names. For example, suppose I have a bunch of SAC files (seismograms) that look as follows:

```
>> ls
```

```
>> HRU.UU.EHZ    NOQ.UU.HHZ    GMU.UU.EHZ    CTU.UU.EHZ
```

The filename convention here looks like: `station_name.network.component`

If I want to make a list of just the station names I could do something like:

```
>> ls *UU* | cut -d. -f1 > stationlist.txt
```

I find myself doing stuff like this all of the time.

3. Homework (5 pts for each question)

To submit your homework, tar your answers into a file called `Lastname_Firstname_HW2.tar` and scp it:

```
>> scp Lastname_Firstname_HW2.tar
youreID@teewinot.warnercnr.colostate.edu:/data/class-data/Fall-
2016/GEOL578/AssignmentDropbox
```

Or, if you are on the WCNR linux network, all you have to do is type

```
cp /path/to/Lastname_Firstname_HW2.tar.gz /data/class-data/Fall-
2016/GEOL578/AssignmentDropbox
```

1) Consider two files given below that each contain a set of Cartesian coordinates. Write an awk script to compute the distance between these pairs of points. Feel free to use any of the other commands we learned in this lecture as well. Hint: to take the square of a quantity in awk, use this sort of format: (\$3-\$2)^2.

	x1	y1			x2	y2	
	0.0	0.0			0.0	0.0	
	0.5	0.1			-0.25	0.1	
	0.75	0.2			-0.5	0.2	
	1.0	0.3			-1.0	0.3	

2) Below is a table of S-wave velocities at the coordinates given by the indicated latitude, and longitude (ϕ) in degrees. Create a file exactly as shown below, and write an awk command that will convert the longitudes given in the file below from the interval: $-180^\circ \leq \phi \leq 180^\circ$ to the interval: $0^\circ \leq \phi \leq 360^\circ$. Note: longitudes from 0° to 180° in the original file should not change. Format your output, such that you have three distinct labeled columns and add a single decimal place to both the latitude and longitude values.

Lon	Lat	dVs
-180	-10	2.3
-135	-10	2.4
-90	-10	2.0
0	10	-0.1
45	10	-0.4
90	10	-1.0
135	10	-1.0
180	10	0.3

3) Consider a file that looks as follows:

Vs
Vp
Rho
Vs
Vp
Rho
Vs

write an awk command that will count the total number of lines that contain the string **Vs**.

4) I have a group of SAC files named as follows:

>> HRU.UU.EHZ NOQ.UU.HHZ GMU.UU.EHZ CTU.UU.EHZ

Using awk, how can we change the names of all of these files so that the EHZ or HHZ is replaced by just Z. So, for example the first file is renamed as: **HRU.UU.Z**. Hint: pipe the output from ls into awk. Also, it may be helpful to first make empty files with these file names, like using:

```
>> echo "" > HRU.UU.EHZ
```

Also, use {print \$1"."\$2".Z"}.

5) Send me your earthquakes.txt file.

6) Use a single line command and pipes to create a file that lists all magnitude ≥ 8.4 earthquakes in the catalog, including Time, Latitude, Longitude, Depth, and Magnitude. Sort the file so that the earliest dates/times are at the top of the file. Send me the file and the command you used to generate it. Hint, look at this page: <http://service.iris.edu/fdsnws/event/1/> and modify the example for minmagnitude=8.5. Add format=text to the URL you use, so that the output is in something you can work with.

awk cheat sheet

get total number of records in a file

```
awk 'END {print NR}'
```

If NR is equal to shell variable 'n' print line

```
awk 'NR == "n" {print $0}'
```

Sum the values along a column (column #2 in this example)

```
awk '{ sum += $2} END {print sum}'
```

Print the sums of the fields of every line

```
awk '{s=0; for (i=1; i<=NF; i++) s=s+$i; print s}'
```

Print out file with double spacing

```
awk '{print ; print " "}'
```

Print fields in reverse order

```
awk '{ for (i = NF; i > 0; --i) print $i }'
```

if else syntax

```
awk '{if ($1 > 2) print $0;
else print $1}' file
```

Concatenate every 5 lines of input, using a comma separator between fields

```
awk 'ORS=NR%5?" , ":"\n"' file
```

5.0 Acknowledgements

This lab is based on an excellent series of computational geophysics write-ups by Michael Thorne of the University of Utah. He's got really cool animations on his website too. <http://web.utah.edu/thorne/>